

Idempotency Middleware

A distributed, exactly-once-execution layer for mutating HTTP requests in Laravel 13.

Project	laravel-idempotency
Stack	Laravel 13 · PHP 8.3 · Redis
Audience	Backend engineers, SRE, security reviewers
Scope	Architecture · request flow · security · operations

This document describes the design and operational behavior of the EnsureIdempotency middleware shipped with this project. It assumes the reader is familiar with HTTP semantics, Laravel middleware, and the Redis data store.

Contents

1.	The Problem	3
2.	Architecture Overview	4
3.	Request Flow — Step by Step	5
4.	Security: OWASP Mapping	9
5.	Behavior Matrix	10
6.	Why Redis	11
7.	Known Limitation	11
8.	Summary	12

1. The Problem

A user taps **Pay \$100**. The request reaches the server, the account is debited, **but the response never reaches the browser** — the network drops, the TCP connection times out, the mobile radio goes to sleep. From the client's perspective the operation appears to have failed; the natural reaction is to retry. Without protection, the user is charged twice.

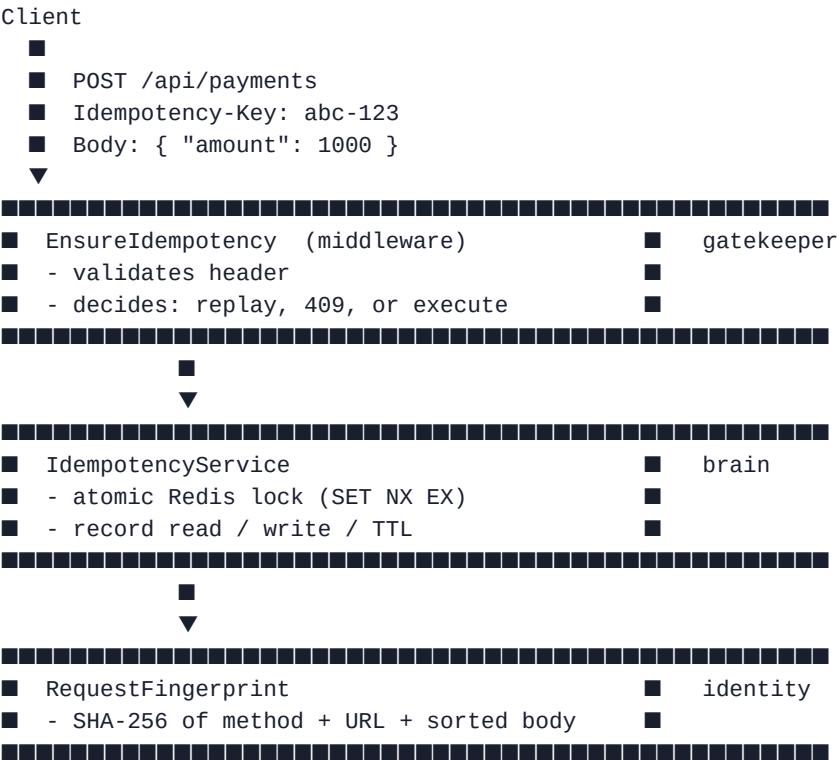
This failure mode is universal in distributed systems and occurs whenever a side-effectful operation is invoked across an unreliable channel:

- Mobile or browser clients reconnecting after a flaky network
- Payment gateways — Stripe, PayPal, Adyen all require Idempotency-Key headers
- Message queues (Kafka, SQS) that guarantee *at-least-once* delivery
- API gateways with automatic retry policies
- Webhook receivers. which the sender redelivers until a 2xx response is observed

Goal. If the same request is delivered N times — by retry, by replay, or by concurrent duplicate submission — the business logic must execute **exactly once**. Every additional attempt must return the cached response from the first execution.

2. Architecture Overview

The system is composed of one HTTP middleware and two services. They cooperate via the Laravel service container and share state through a Redis-backed cache.



Component Responsibilities

Component	File	Responsibility
EnsureIdempotency	app/Http/Middleware/EnsureIdempotency.php	HTTP-level orchestration: header validation, fingerprint comparison, lock acquisition
IdempotencyService	app/Services/Idempotency/IdempotencyService.php	Cache service, atomic withLock() critical section, key scoping by user, key-fingerprint
RequestFingerprint	app/Services/Idempotency/RequestFingerprint.php	Deterministic SHA-256 fingerprint over method, URL, query, and recursive body
IdempotencyRecord	app/Services/Idempotency/IdempotencyRecord.php	Immutable DTO. Holds the cached status, body, headers, fingerprint, user
Service provider	app/Providers/IdempotencyServiceProvider.php	Binds services as singletons, merges config, wires the configured log channels
Config	config/idempotency.php	TTL, lock ttl/wait, cache store, key regex, cacheable status range, max body size

3. Request Flow — Step by Step

The middleware processes a mutating request through nine well-defined stages. Each stage is fail-fast — invalid inputs are rejected with a structured JSON error before any cache or downstream operation is attempted.

Step 1 — Incoming Request

```
POST /api/payments HTTP/1.1
Host: api.example.com
Idempotency-Key: order-9c8a-2026-06-14
Content-Type: application/json
Authorization: Bearer ...
```

```
{ "amount": 100000, "currency": "USD" }
```

Step 2 — Header Validation

The middleware runs four short-circuiting checks in order:

Check	On failure	Why it matters
Method is POST/PUT/PATCH/DELETE	405 Method not allowed	GET/HEAD are already idempotent by definition
Idempotency-Key header present	400 idempotency_key_required	Without a key, duplicates cannot be identified
Key matches strict regex	400 idempotency_key_invalid	Defends against injection — value flows into the Redis key
Resolve authenticated user (if any)	null → scope key with 'anon'	Per-user isolation (OWASP A01)

Step 3 — Request Fingerprint

A SHA-256 hash is computed from the canonical representation of the request. JSON keys are recursively ksorted so semantically equal payloads always hash to the same value.

```
$payload = [
    'method' => 'POST',
    'url'     => 'http://api.example.com/api/payments',
    'body'    => ['amount' => 100000, 'currency' => 'USD'], // ksorted
];
$fingerprint = hash('sha256', json_encode($payload));
// → "8f3a7b..." (64 hex chars)
```

The fingerprint is the second-line defense against an adversary or buggy client reusing the same key with a different payload. The first-line defense is the regex; the fingerprint catches anything that gets past it (OWASP A08).

Step 4 — Fast Path (pre-lock cache lookup)

Before any lock is taken, the middleware probes Redis directly. The overwhelming majority of retries in production happen *after* the original response has already been stored — paying for a lock acquisition in that case is wasteful.

- Fingerprint matches the cached record → replay with header **Idempotent-Replayed: true**

- Fingerprint differs → **409 Conflict**
- No record found → proceed to locked path

Step 5 — Atomic Lock

If the fast path missed, the request is genuinely first-time *or* arriving concurrently with N other duplicates. `Cache::lock()` — which under Redis compiles to **SET NX EX** — guarantees that exactly one worker out of the N wins the race; the remaining workers block.

```
$lock = $this->cache()->lock($lockKey, $lockTtl);
$lock->block($lockWait); // wait up to N seconds for the original to finish
// only one execution path reaches this line at a time
```

Any worker that cannot acquire the lock within the configured wait window receives **503 Service Unavailable** with a **Retry-After** header — a deliberate, observable signal rather than a silent timeout.

Step 6 — Double-Checked Read

The instant a worker holds the lock, it re-reads the cache. This is the subtle but essential step that closes the race window between fast-path miss and lock acquisition:

```
t=0   Request A acquires lock, starts controller
t=10  Request B arrives; fast-path miss; blocks on lock
t=15  Request A finishes, writes Redis, releases lock
t=15  Request B acquires lock
t=15  Without double-check, B would run the controller too
      With double-check, B sees A's record and replays.
```

This is the same Double-Checked Locking idiom used for singleton initialization in Java and C++.

Step 7 — Execute the Controller

Only now is `$next($request)` invoked. Whatever the controller does — DB writes, third-party API calls, queue dispatches — runs **at most once** per (user, key) pair. This is the exactly-once guarantee.

Step 8 — Persist the Response

The controller's response is stored in Redis only if both of the following hold:

Condition	Rationale
Status code in [200, 499]	5xx indicates a transient server fault — the client must remain free to legitimately retry
Body size ≤ max_body_bytes (default 1 MiB)	Bounds Redis memory consumption per record

The stored record has this shape:

```
{
  "key": "order-9c8a-2026-06-14",
  "user_id": 42,
  "fingerprint": "8f3a7b...",
  "status": 201,
  "response_body": "{\"id\":\"01H...\",\"amount\":100000,...}",
  "headers": { "content-type": "application/json" },
  "created_at": 1750000000,
  "expires_at": 1750086400
}
```

```
}
```

Step 9 — Release the Lock

Release is performed in a finally block, so even an exception from the controller cannot leak a held lock. If release itself fails (e.g. the lock has already auto-expired under load), the error is swallowed — the lock TTL is the ultimate backstop and guarantees there is no deadlock.

4. Security: OWASP Mapping

The middleware actively mitigates six items from the OWASP Top 10. Each mitigation is enforced in code, not by convention.

Weakness	Mitigation
A01 — Broken Access Control	The Redis cache key embeds user_id: <code>idem:{userId}:{key}</code> . Two users using
A02 — Cryptographic Failures	Raw request bodies are never persisted — only a SHA-256 fingerprint is. Fingerprint compar
A03 — Injection	The header value is validated against a strict allow-list regex <code>[A-Za-z0-9_\-:]{8,255}</code>
A07 — Identification & Auth Failures	Records are bound to <code>\$request->user()->getAuthIdentifier()</code> . A stolen token f
A08 — Software & Data Integrity Failures	Bounded TTL plus fingerprint-mismatch detection blocks replay attacks: an attacker who rec
A09 — Security Logging & Monitoring Failures	Every lifecycle event is logged (stored, replay, fingerprint_conflict, lock_timeout, invalid_key,

5. Behavior Matrix

The table below enumerates every scenario the middleware handles and the observable outcome for the client.

Scenario	Outcome
First use of a key	Controller executes; response stored; HTTP 201 (or whatever the controller returned)
Same key + identical body	Cached response replayed; <code>Idempotent-Replayed: true</code> header
Same key + different body	409 Conflict — <code>idempotency_key_conflict</code>
500 concurrent requests, same key	Exactly one execution; 499 byte-identical replays
Lock wait exceeded (default 10 s)	503 + <code>Retry-After: 10</code>
Malformed key (regex fail)	400 — <code>idempotency_key_invalid</code>
Header missing on mutating verb	400 — <code>idempotency_key_required</code>
Key reused after TTL expiry	Treated as a brand-new request
DELETE then immediate DELETE replay	Same 200 response — never a 404 race
Controller returns 5xx	Not cached — client may legitimately retry
Response body > <code>max_body_bytes</code>	Not cached — controller still executes

6. Why Redis

Three properties of Redis make it the right backing store for this workload:

Property	Why it matters here
Sub-millisecond round-trip	Every request pays two-to-three cache operations (lock + read + write). On a relational DB the same path
Native TTL	Records self-expire after the configured window. No background job, no scheduler — the same property
Atomic SET NX EX	A single network round-trip yields a correct distributed lock. The DB equivalent would be advisory locks (s

7. Known Limitation

The middleware **cannot** protect against a crash that occurs after the controller has committed to the database but before the response has been written to Redis. A subsequent retry will find no cached record and execute the controller a second time.

The complete remedy is the **Outbox Pattern**: persist the idempotency record in the same database transaction as the business write, then asynchronously replicate it to the cache. That is significantly more complex and is intentionally out of scope for this implementation. For the majority of workloads — non-financial mutations, retry windows under the TTL, monitored deployments — the current level of protection is sufficient.

8. Summary

One sentence: this middleware ensures that even if a client or its network delivers the same mutating request 500 times concurrently, the business operation executes exactly once and the other 499 attempts receive byte-identical replays — with end-to-end protection against race conditions, replay attacks, and cross-user response leakage.

Operational highlights:

- **Storage:** Redis, 24-hour TTL by default, user-scoped keys
- **Concurrency:** `Cache::lock()` (Redis SET NX EX) + double-checked read
- **Integrity:** SHA-256 fingerprint + constant-time compare
- **Observability:** structured events without payload leakage
- **Wiring:** opt-in per route via the idempotent middleware alias

— End of document —